Árvore Binária de Busca

Estruturas de Dados



UnB | CIC

Definição

Uma Árvore Binária de Busca (ABB) é uma estrutura de dados hierárquica.

Cada nó possui:

- Uma chave.
- Um valor associado.
- Um nó filho esquerdo.
- Um nó filho direito.

Propriedades dos nós:

- Todos os valores no filho esquerdo são menores que a chave do nó.
- Todos os valores no filho direito são maiores que a chave do nó.

Propriedades

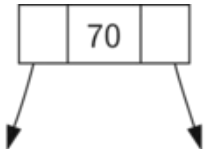
- É usada em aplicações como dicionários, índices e sistemas de busca.
- Permite operações eficientes de busca, inserção e remoção.
- Altura da árvore influencia a eficiência:
  - Melhor caso: altura O(log n) (árvore balanceada).
  - Pior caso: O(n) (árvore degenerada)

Exemplo

- Nós inseridos na ordem:

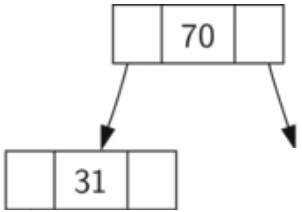
Exemplo

- Nós inseridos na ordem: 70,



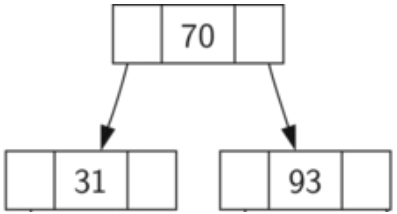
Exemplo

- Nós inseridos na ordem: 70, 31,



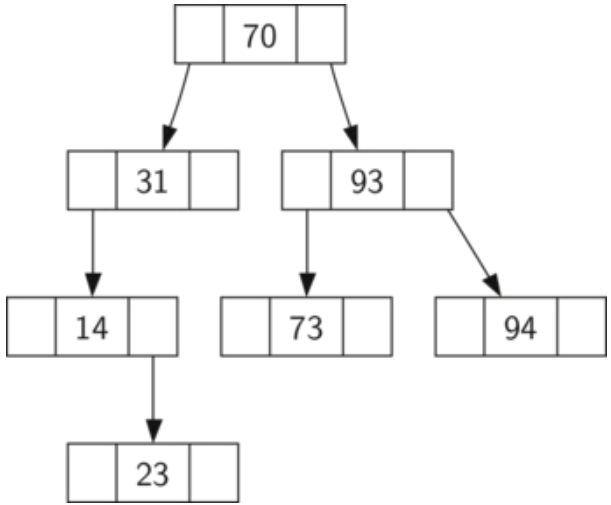
Exemplo

- Nós inseridos na ordem: 70, 31, 93,



Exemplo

- Nós inseridos na ordem: 70, 31, 93, 94, 14, 23 e 73

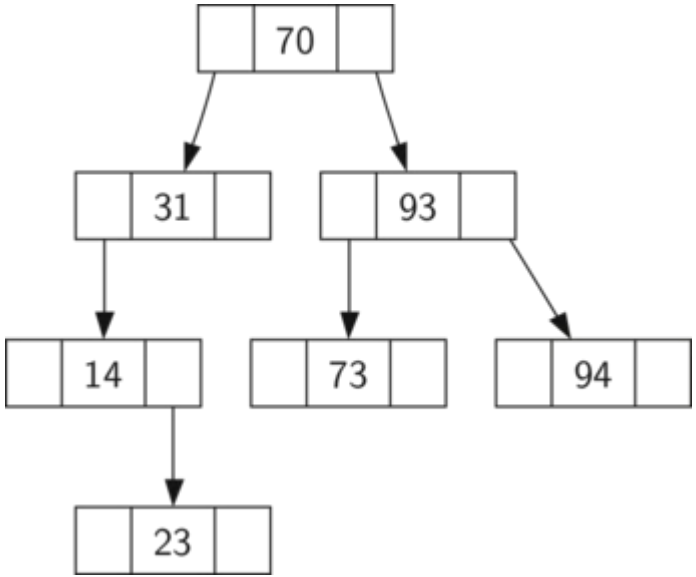


Tipo Abstrato de Dados BST

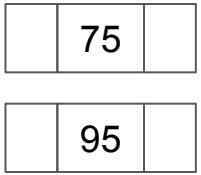
```
class TreeNode:

    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.left = None
        self.right = None
```

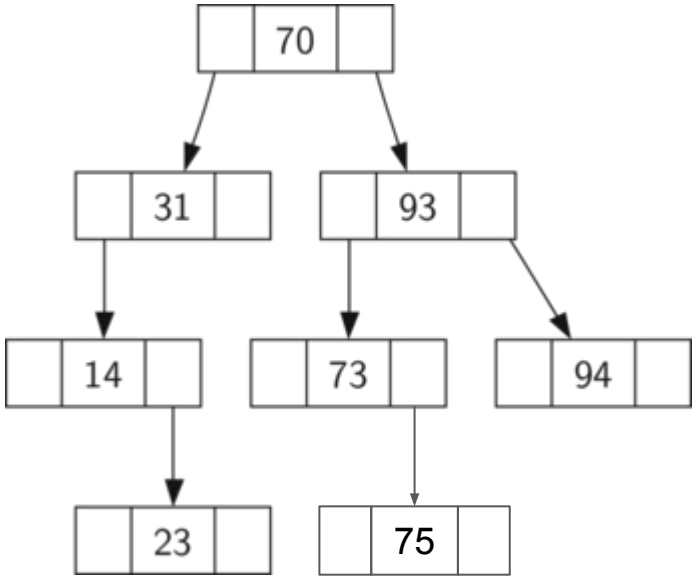
Inserção



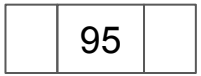
Inserção dos nós:



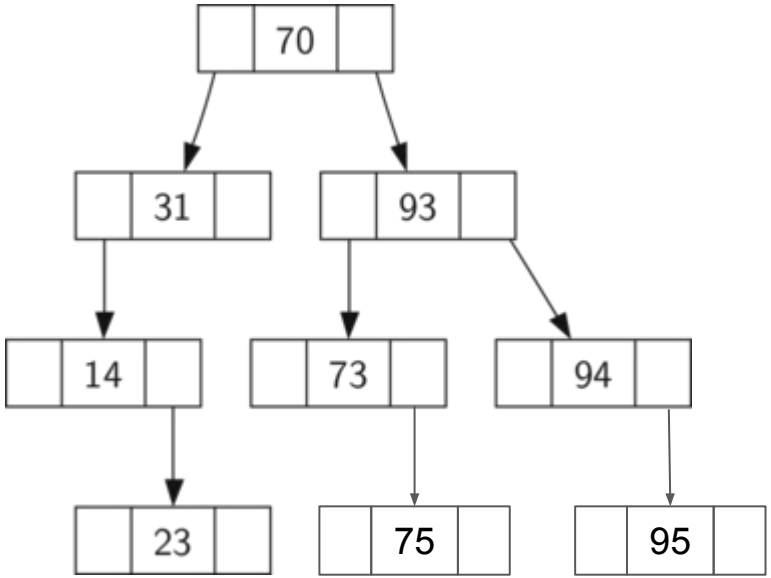
Inserção



Inserção dos nós:



Inserção



Inserção dos nós:

Inserção

```
class TreeNode:
    ...
    def insert(self, key, value):
        if key < self.key:
            if self.left:
                self.left.insert(key, value)
            else:
                self.left = TreeNode(key, value)
        elif key > self.key:
            if self.right:
                self.right.insert(key, value)
            else:
                self.right = TreeNode(key, value)
```

Inserção

```
class TreeNode:
    ...
    def __setitem__(self, key, value):
        self.insert(key, value)
```

Inserção

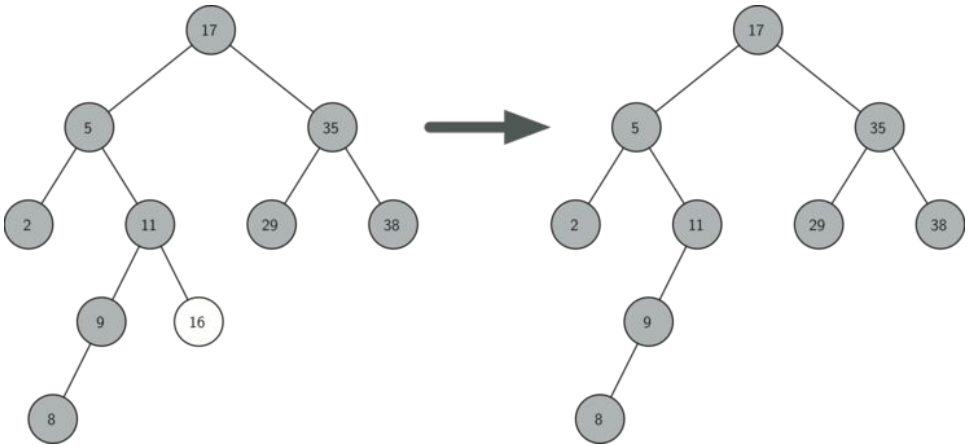
```
class TreeNode:
    ...
    def __setitem__(self, key, value):
        self.insert(key, value)
```

Permite manipulações como
`bst[10] = 'ola mundo'`

Deleção

Três casos base de deleção:

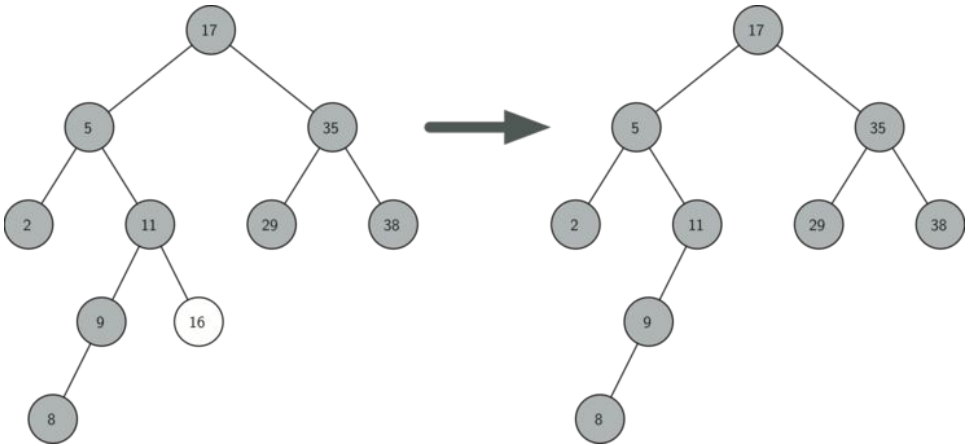
- O nó a ser excluído não tem filhos.



Deleção

Três casos base de deleção:

- O nó a ser excluído não tem filhos.

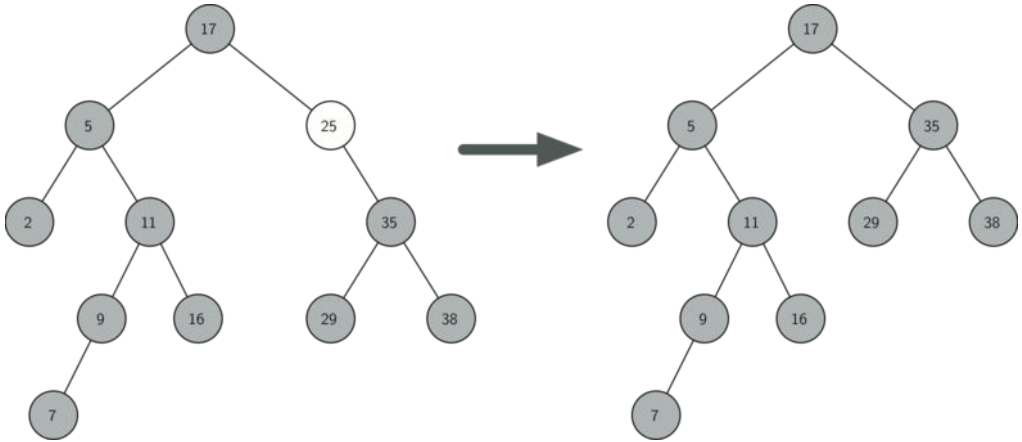


1. Basta remover o nó.
2. Atualize a referência do pai para que ele não aponte mais para o nó excluído.

Deleção

Três casos base de deleção:

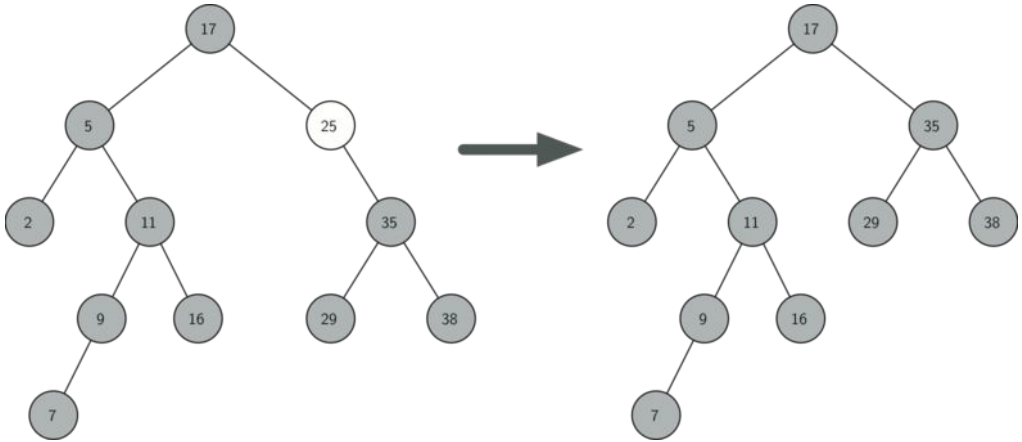
- O nó a ser excluído tem apenas um filho.



Deleção

Três casos base de deleção:

- O nó a ser excluído tem apenas um filho.

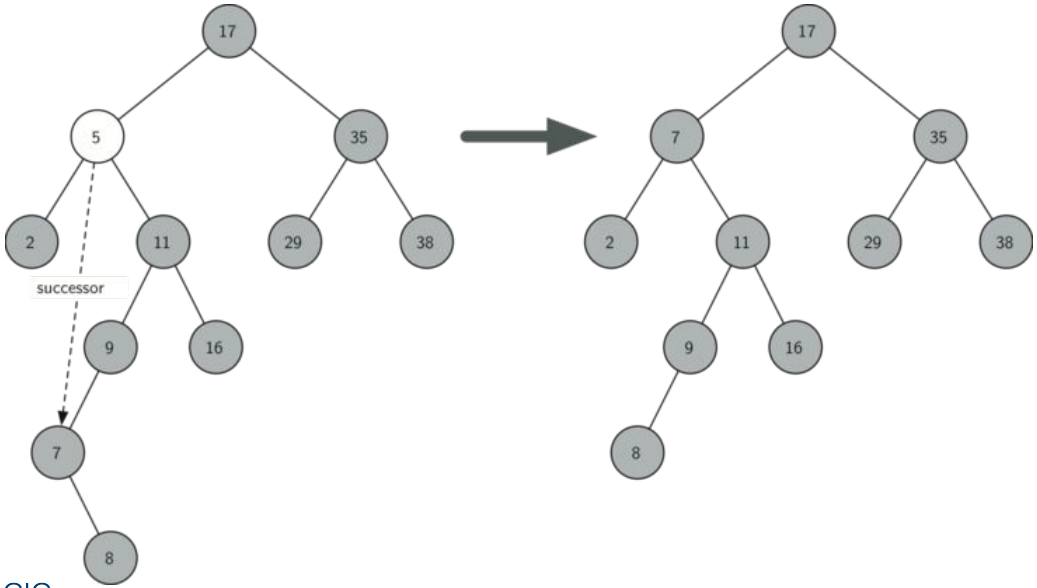


1. Substitua o nó pelo seu único filho.
2. Atualize a referência do pai para apontar para o filho do nó excluído.

Deleção

Três casos base de deleção:

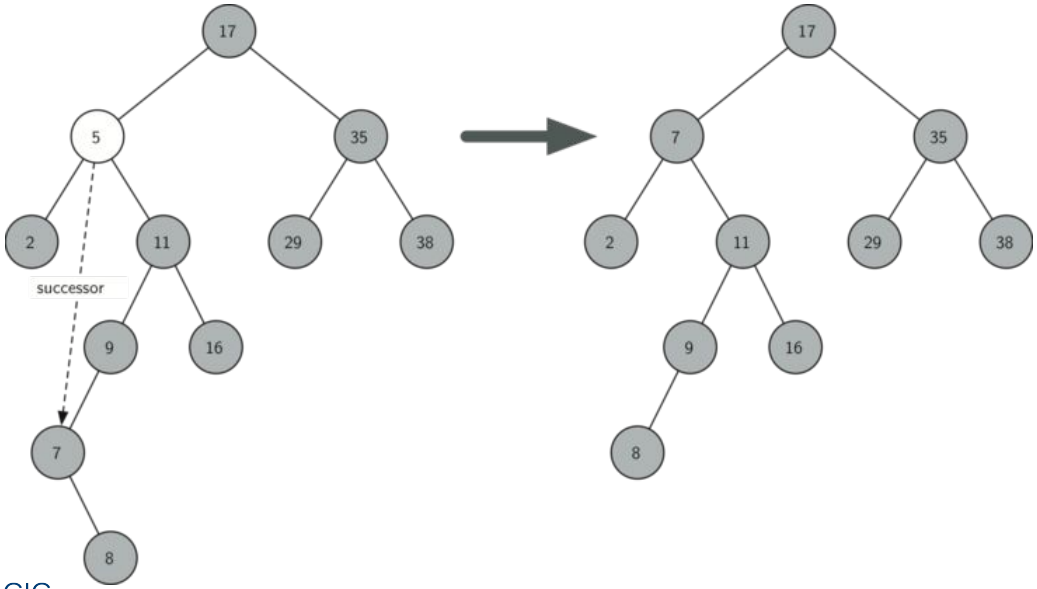
- O nó a ser excluído tem dois filhos.



Deleção

Três casos base de deleção:

- O nó a ser excluído tem dois filhos.



1. Encontre o sucessor do nó (o menor valor na subárvore direita).
2. Substitua os dados (chave e valor) do nó excluído pelos dados do sucessor.
3. Recursivamente, remova o sucessor da sua posição original (ele será um caso de exclusão de nó folha ou com um filho, porque o sucessor é o menor nó de sua subárvore).

Deleção

```
class TreeNode:
    ...
    def delete(self, key):
        if key < self.key and self.left:
            self.left = self.left.delete(key)
        elif key > self.key and self.right:
            self.right = self.right.delete(key)
        else:
            if not self.left and not self.right:
                return None
            if not self.left:
                return self.right
            if not self.right:
                return self.left
            min_larger_node = self.right.find_min()
            self.key, self.value = min_larger_node.key, min_larger_node.value
            self.right = self.right.delete(min_larger_node.key)
        return self
```

Deleção

```
class TreeNode:
    ...
    def delete(self, key):
        if key < self.key and self.left:
            self.left = self.left.delete(key)
        elif key > self.key and self.right:
            self.right = self.right.delete(key)
        else:
            if not self.left and not self.right:
                return None
            if not self.left:
                return self.right
            if not self.right:
                return self.left
            min_larger_node = self.right.find_min()
            self.key, self.value = min_larger_node.key, min_larger_node.value
            self.right = self.right.delete(min_larger_node.key)
        return self
```

O nó a ser excluído não tem filhos.

O nó a ser excluído tem apenas um filho.

O nó a ser excluído tem dois filhos.

Deleção

```
class TreeNode:
    ...
    def find_min(self):
        current = self
        while current.left:
            current = current.left
        return current
```

Busca

```
class TreeNode:
    ...
    def search(self, key):
        if key == self.key:
            return self.value
        elif key < self.key and self.left:
            return self.left.search(key)
        elif key > self.key and self.right:
            return self.right.search(key)
        return None
```

Busca

```
class TreeNode:
    ...
    def __getitem__(self, key):
        return self.search(key)

    def __contains__(self, key):
        return self.search(key) is not None
```

Busca

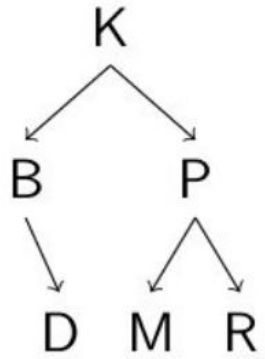
```
class TreeNode:
    ...
    def __getitem__(self, key):
        return self.search(key)

    def __contains__(self, key):
        return self.search(key) is not None
```

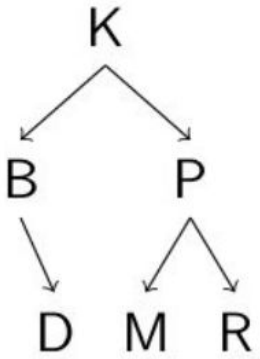
Permite usar a sintaxe de colchetes para acessar valores:
`valor = bst[12]`

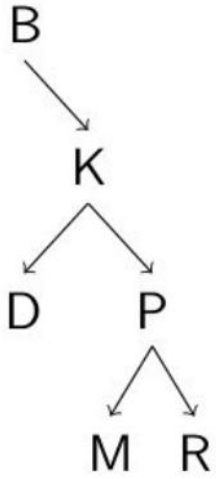
Permite usar o operador `in`
`12 in bst`

Busca eficiente?

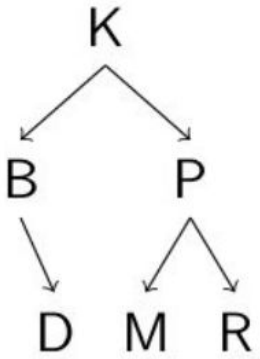


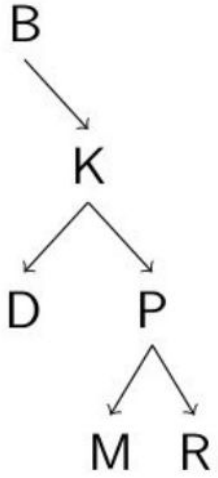
Busca eficiente?

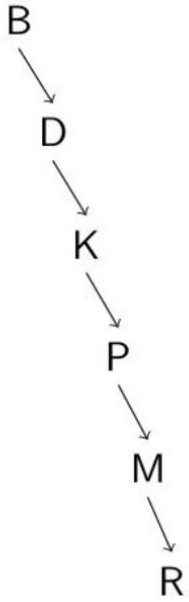




Busca eficiente?







Busca eficiente

- O custo da busca é diretamente proporcional à altura da árvore.
- É desejável que os elementos da árvore estejam dispostos da forma mais compacta possível (de menor altura).
- Quanto menor a diferença entre as alturas das subárvores, melhor a distribuição dos elementos na estrutura.

ABB Balanceada

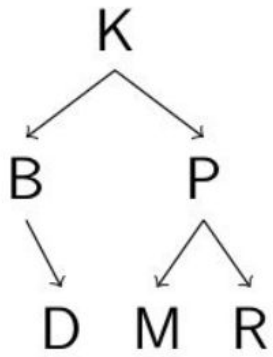
Uma árvore binária é considerada balanceada se:

- A diferença de altura entre as subárvores esquerda e direita de qualquer nó for no máximo 1.
- Ambas as subárvores também forem balanceadas.

ABB Balanceada

Uma árvore binária é considerada balanceada se:

- A diferença de altura entre as subárvores esquerda e direita de qualquer nó for no máximo 1.
- Ambas as subárvores também forem balanceadas.



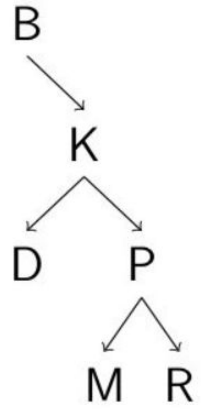
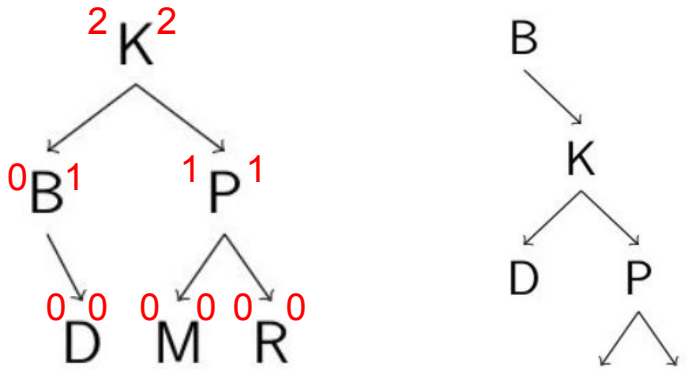


ABB Balanceada

Uma árvore binária é considerada balanceada se:

- A diferença de altura entre as subárvores esquerda e direita de qualquer nó for no máximo 1.
- Ambas as subárvores também forem balanceadas.



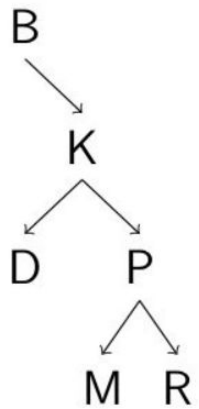
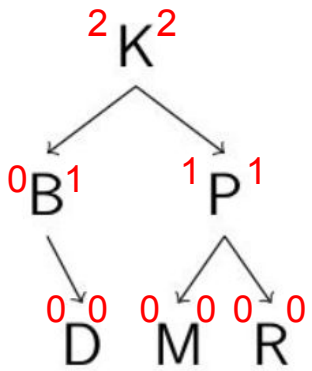


ABB Balanceada

Uma árvore binária é considerada balanceada se:

- A diferença de altura entre as subárvores esquerda e direita de qualquer nó for no máximo 1.
- Ambas as subárvores também forem balanceadas.



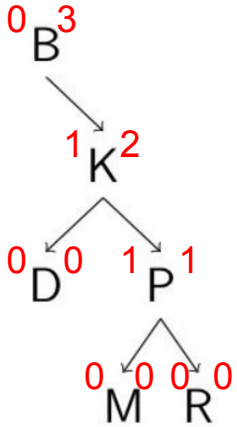
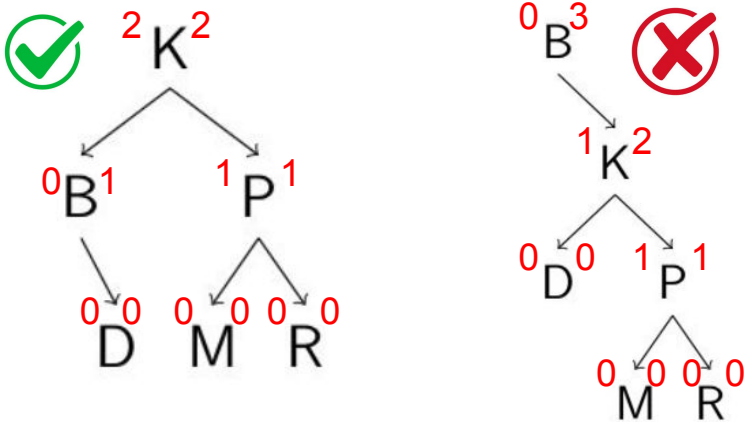


ABB Balanceada

Uma árvore binária é considerada balanceada se:

- A diferença de altura entre as subárvores esquerda e direita de qualquer nó for no máximo 1.
- Ambas as subárvores também forem balanceadas.



Algoritmo de Verificação de Balanceamento

1. Percorrer a árvore em pós-ordem (subárvores antes do nó atual).
2. Para cada nó:
   a. Calcular a altura da subárvore esquerda.
   b. Calcular a altura da subárvore direita.
   c. Verificar a diferença entre as alturas.
3. Se a diferença de altura for maior que 1 em qualquer nó, a árvore não é balanceada.

Calcula a Altura

```
def height(node):
    if node is None:
        return -1

    left_height = height(node.left)
    right_height = height(node.right)
    return max(left_height, right_height) + 1
```

Verifica o Balanceamento

```
def is_balanced(node):
    if node is None:
        return True

    left_height = calculate_height(node.left)
    right_height = calculate_height(node.right)

    if abs(left_height - right_height) > 1:
        return False

    return is_balanced(node.left) and is_balanced(node.right)
```

Balanceamento Estático

O balanceamento é feito uma única vez, geralmente durante a construção da árvore.

Após a árvore ser criada, ela não se ajusta automaticamente a alterações.

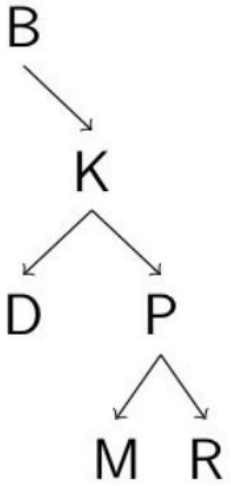
Características:

- Usado quando os dados são conhecidos antecipadamente e raramente mudam.
- Uma árvore balanceada estaticamente pode ser construída a partir de uma lista ordenada de elementos.

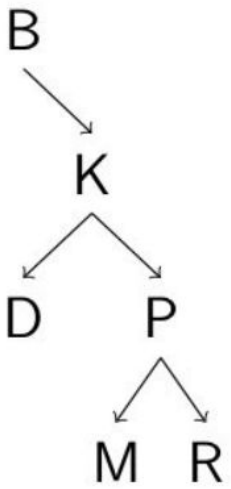
Vantagem: Simples e eficiente para cenários de leitura fixa.

Desvantagem: Se os dados forem alterados frequentemente, o balanceamento será perdido.

Balanceamento Estático



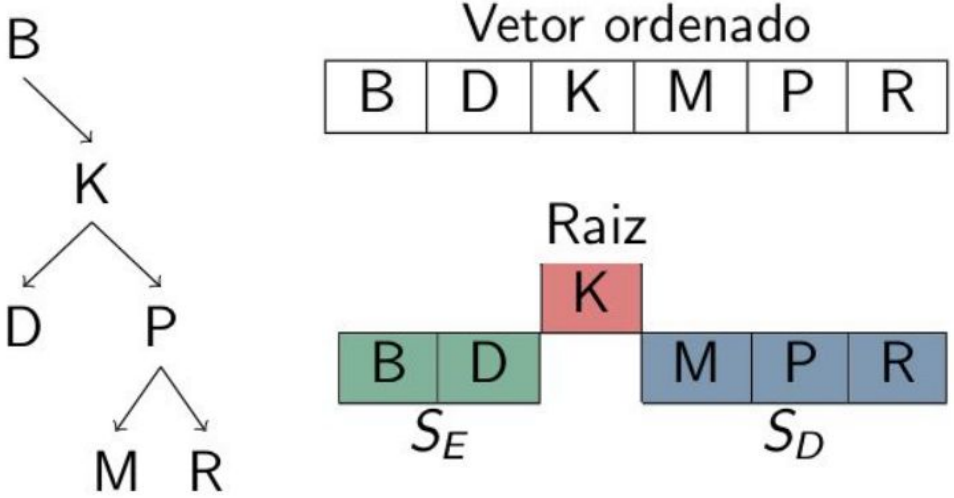
Balanceamento Estático



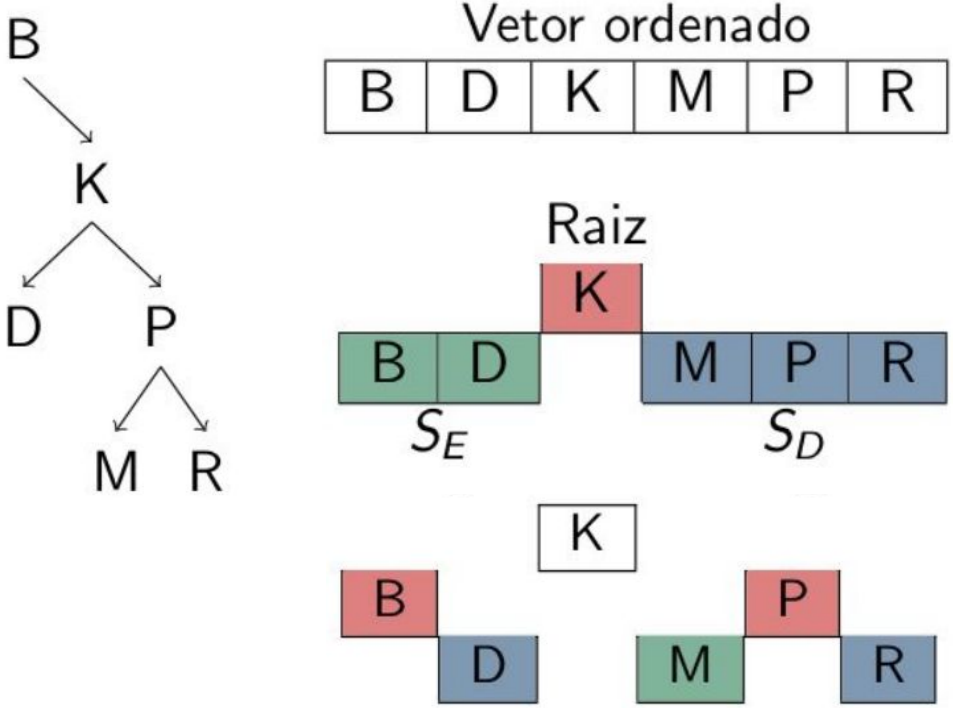
Vetor ordenado

B	D	K	M	P	R

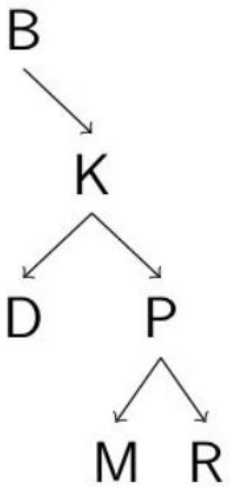
Balanceamento Estático

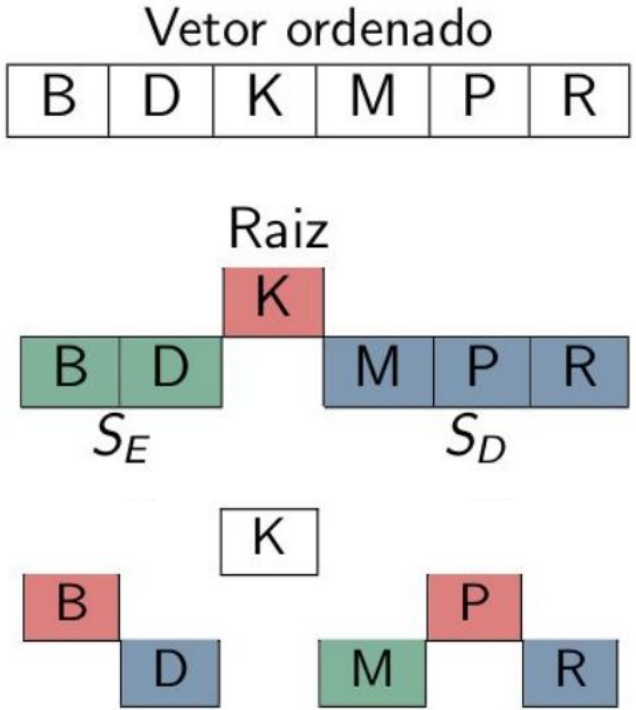


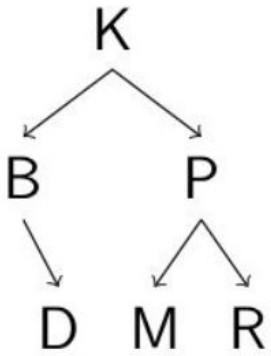
Balanceamento Estático



Balanceamento Estático







Algoritmo de Balanceamento Estático

```
def build_tree(alist):
    if not alist:
        return None

    mid = len(alist) // 2
    root = TreeNode(alist[mid], alist[mid])
    root.left = build_tree(alist[:mid])
    root.right = build_tree(alist[mid + 1:])
    return root
```

Balanceamento Dinâmico

A árvore se ajusta automaticamente em resposta a inserções e remoções para manter o balanceamento.

Características:

- Utiliza algoritmos específicos (ex.: rotações) para corrigir o balanceamento sempre que necessário.
- Árvores como AVL e Red-Black são exemplos de estruturas com balanceamento dinâmico.

Vantagem: Adequado para cenários com modificações frequentes.

Desvantagem: Um pouco mais complexo de implementar e pode introduzir sobrecarga.

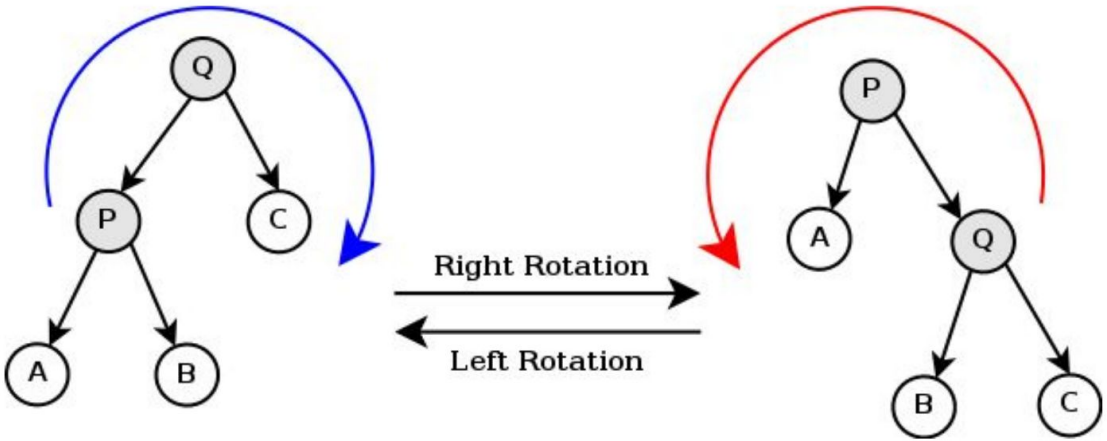
Rotações

Uma rotação é uma operação que rearranja os nós da árvore para corrigir um desequilíbrio.

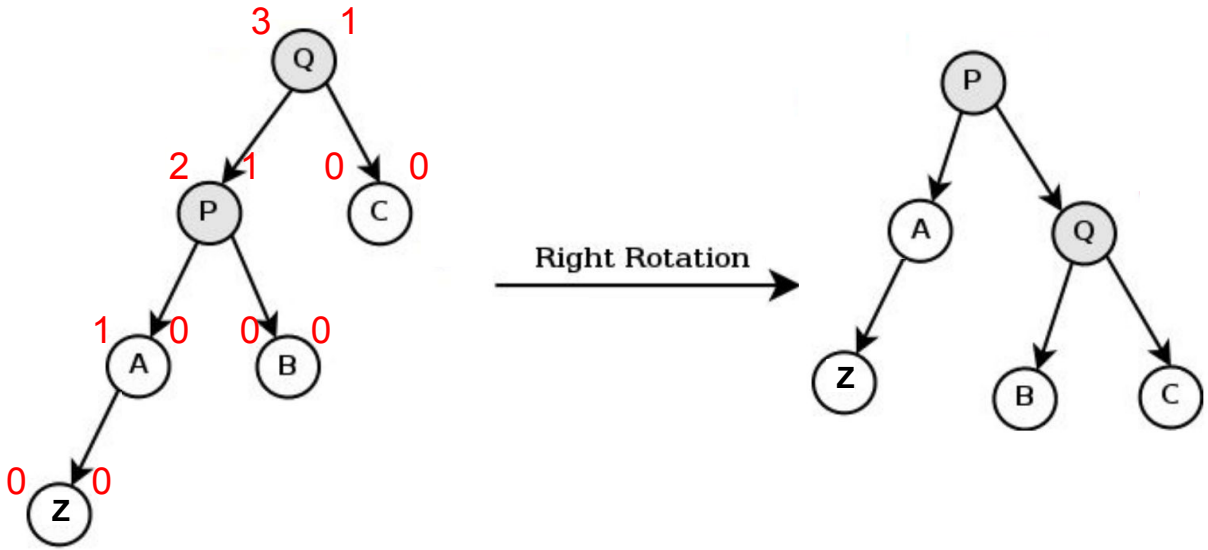
Existem duas rotações principais:

- **Rotação Simples:** Usada quando o desequilíbrio é linear (esquerda-esquerda ou direita-direita).
- **Rotação Dupla:** Usada quando o desequilíbrio é mais complexo (esquerda-direita ou direita-esquerda).

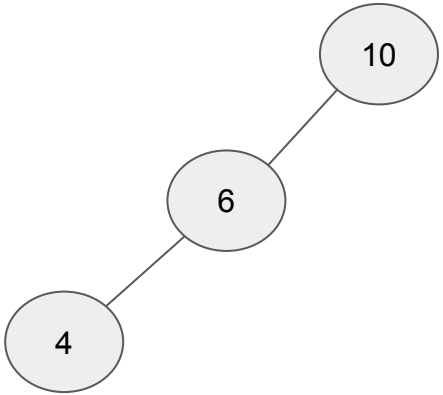
Rotação Simples



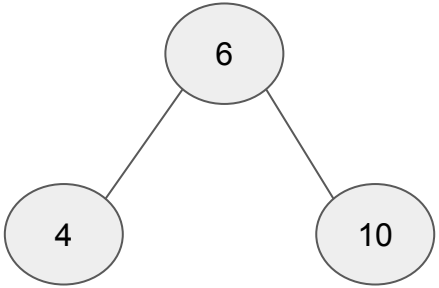
Rotação Simples



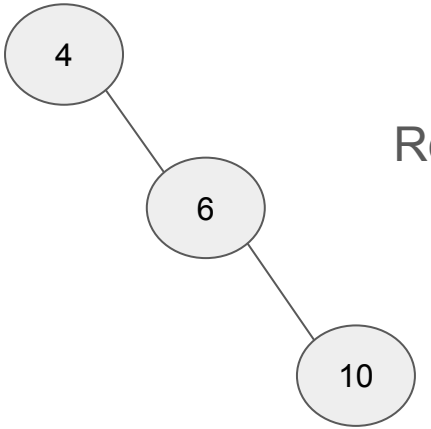
Rotação Simples



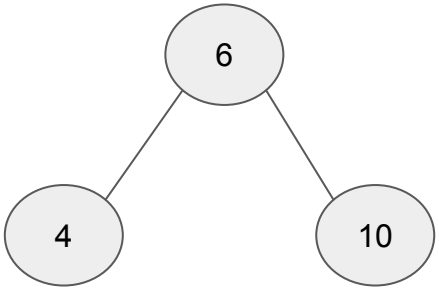
Rotação à direita



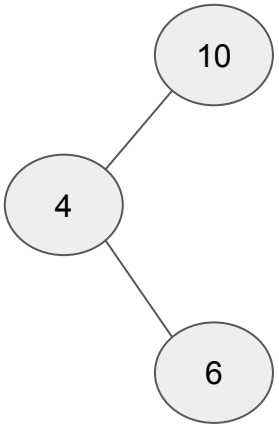
Rotação Simples



Rotação à esquerda

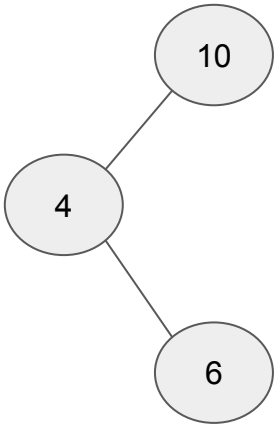


Rotação Dupla

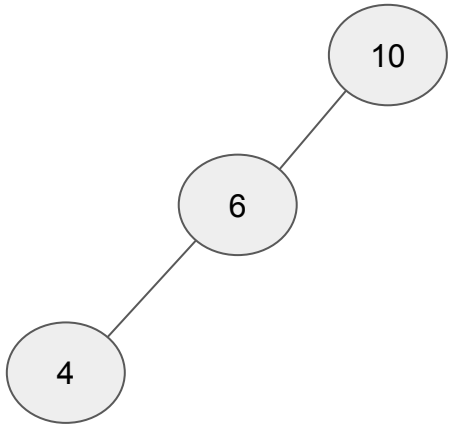


Rotação à esquerda
+ Rotação à direita

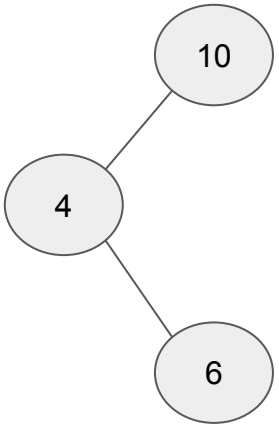
Rotação Dupla



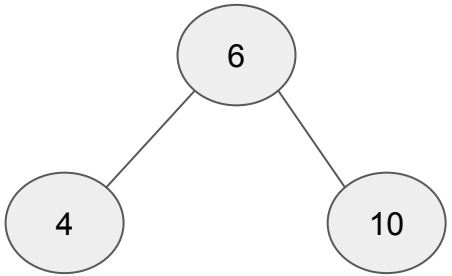
Rotação à esquerda
+ Rotação à direita



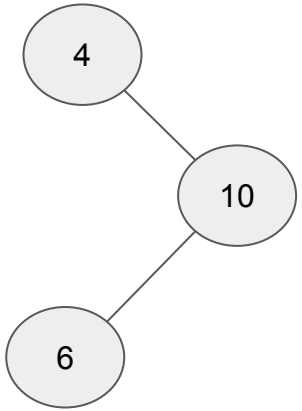
Rotação Dupla



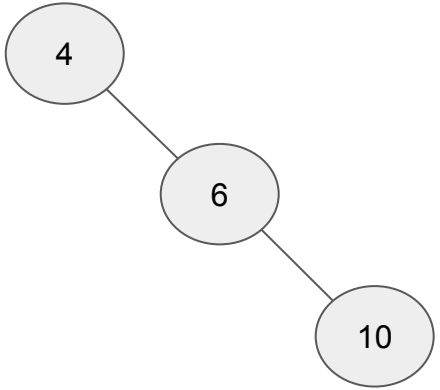
Rotação à esquerda
+ Rotação à direita



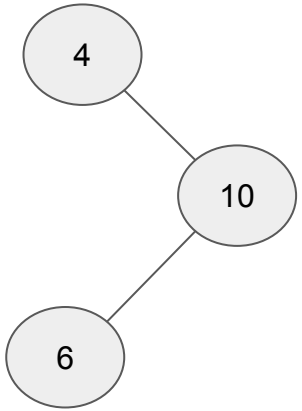
Rotação Dupla



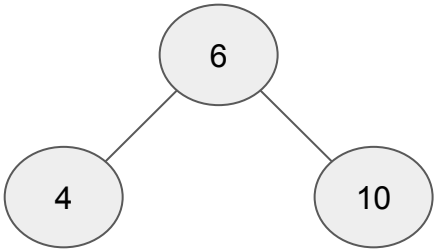
Rotação à direita +
Rotação à esquerda



Rotação Dupla

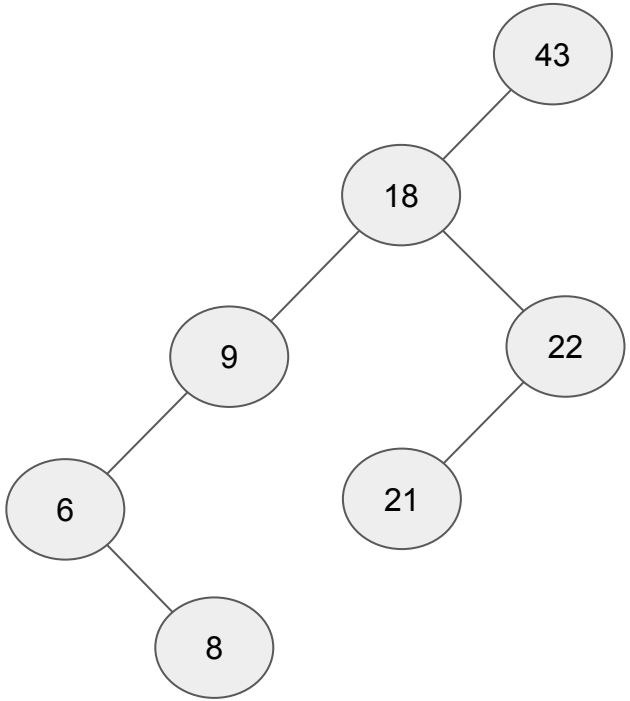


Rotação à direita +
Rotação à esquerda



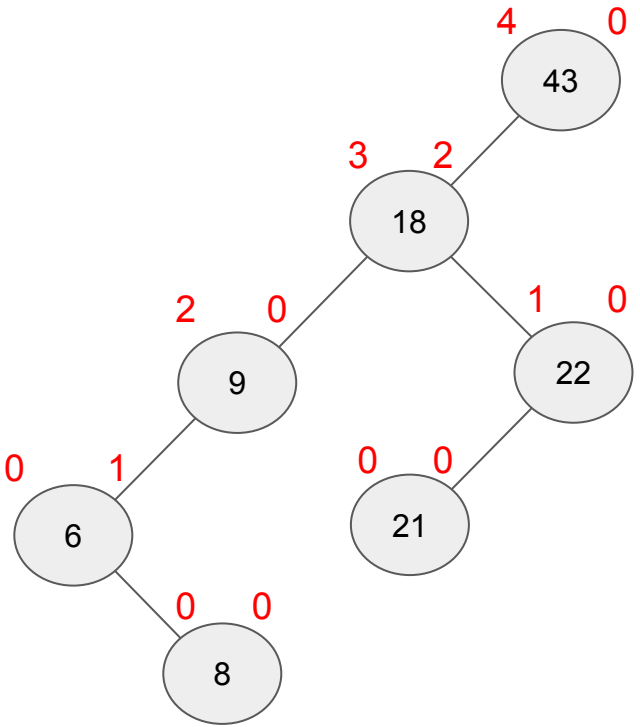
Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6 e 8
- Sem balanceamento:



Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6 e 8
- Sem balanceamento:



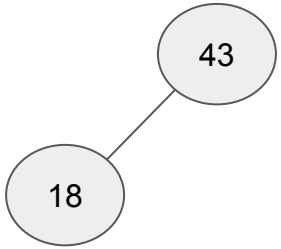
Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43
- Com balanceamento:



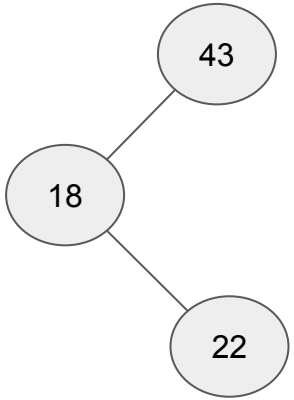
Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18
- Com balanceamento:



Balanceamento Dinâmico por Rotação

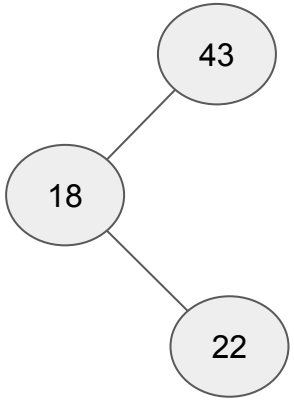
- Nós inseridos na ordem: 43, 18, 22
- Com balanceamento:



Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22
- Com balanceamento:

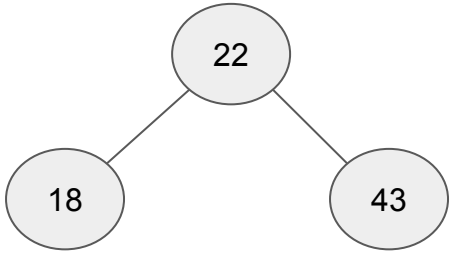
Rotação à esquerda
+ Rotação à direita



Balanceamento Dinâmico por Rotação

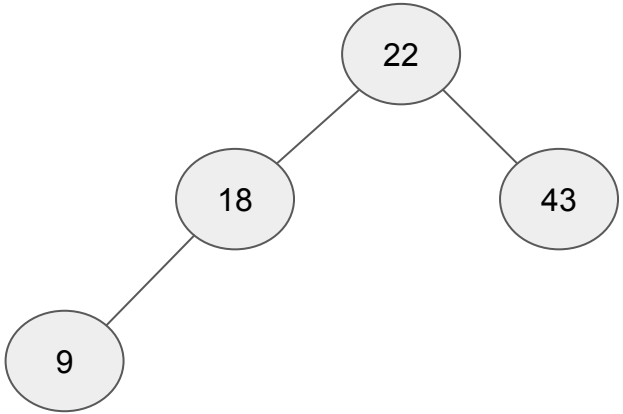
- Nós inseridos na ordem: 43, 18, 22
- Com balanceamento:

Rotação à esquerda
+ Rotação à direita



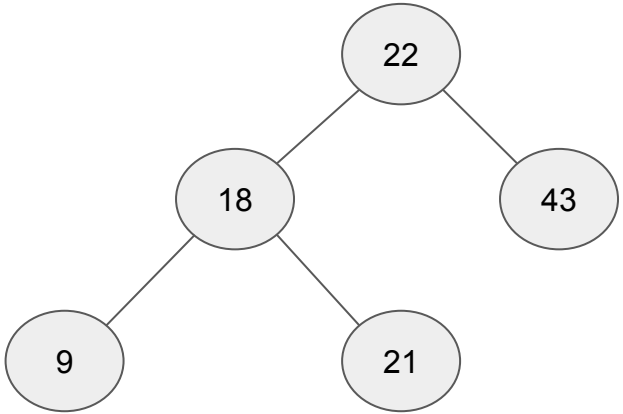
Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9
- Com balanceamento:



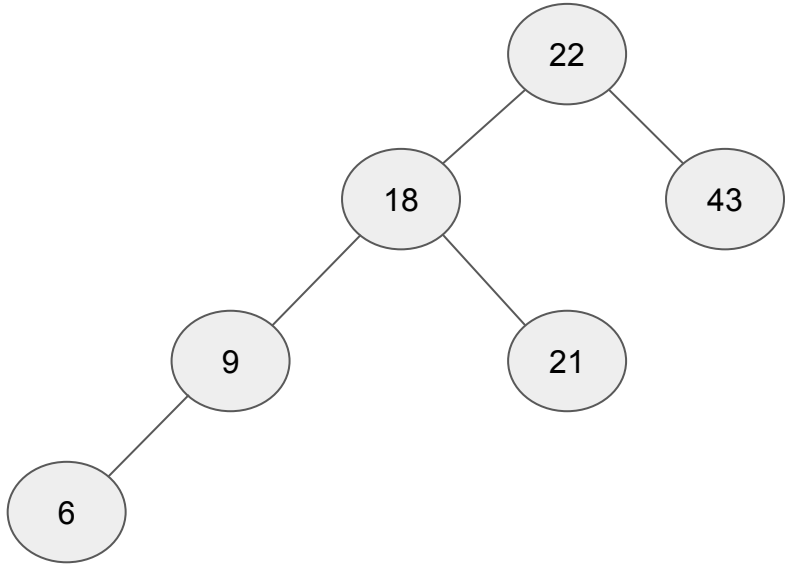
Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9, 21
- Com balanceamento:



Balanceamento Dinâmico por Rotação

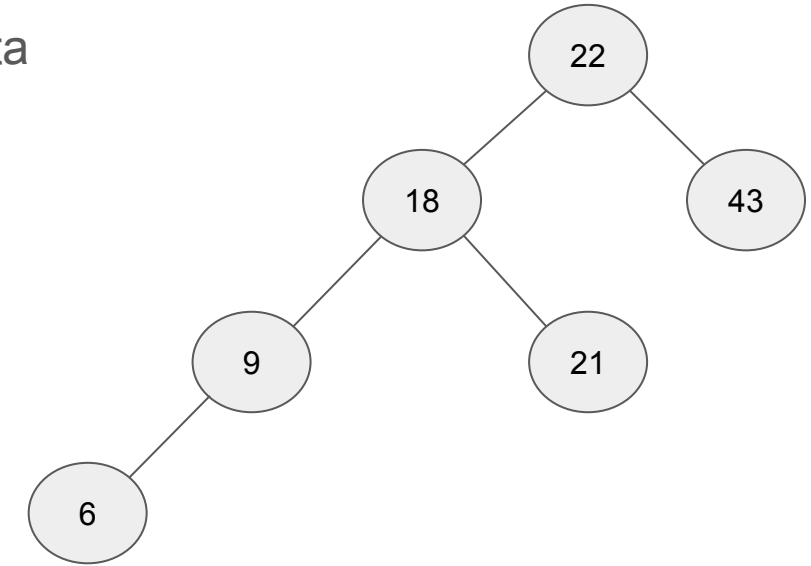
- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6
- Com balanceamento:



Balanceamento Dinâmico por Rotação

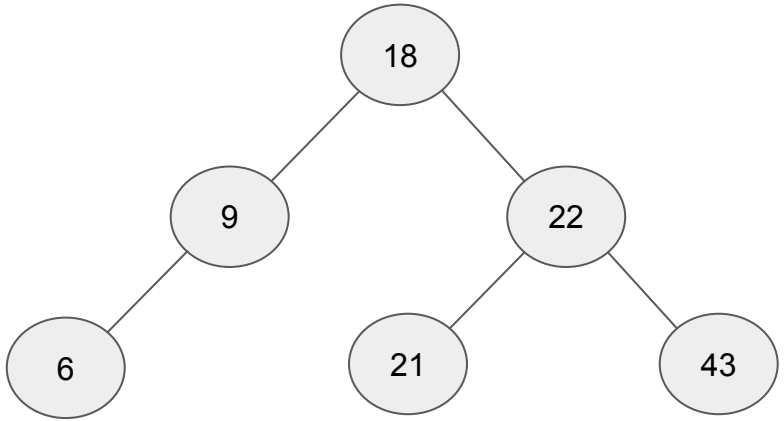
- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6
- Com balanceamento:

Rotação à direita



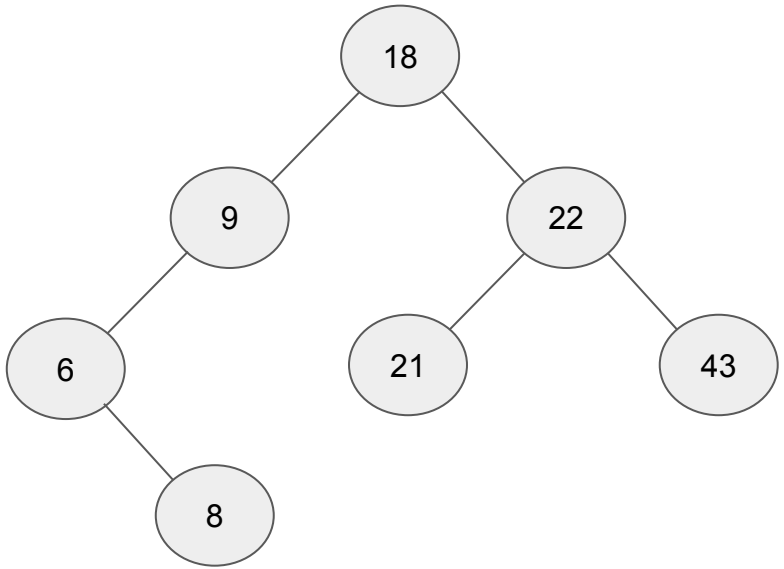
Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6
- Com balanceamento:



Balanceamento Dinâmico por Rotação

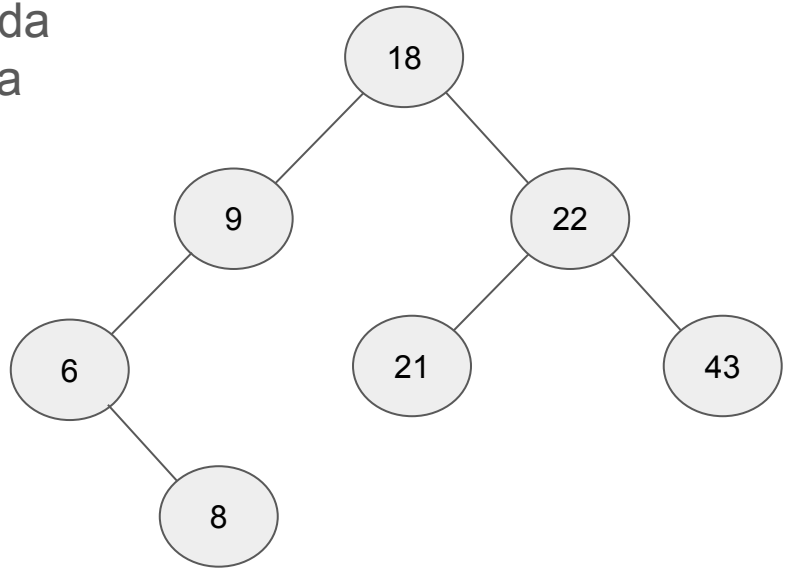
- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6 e 8
- Com balanceamento:



Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6 e 8
- Com balanceamento:

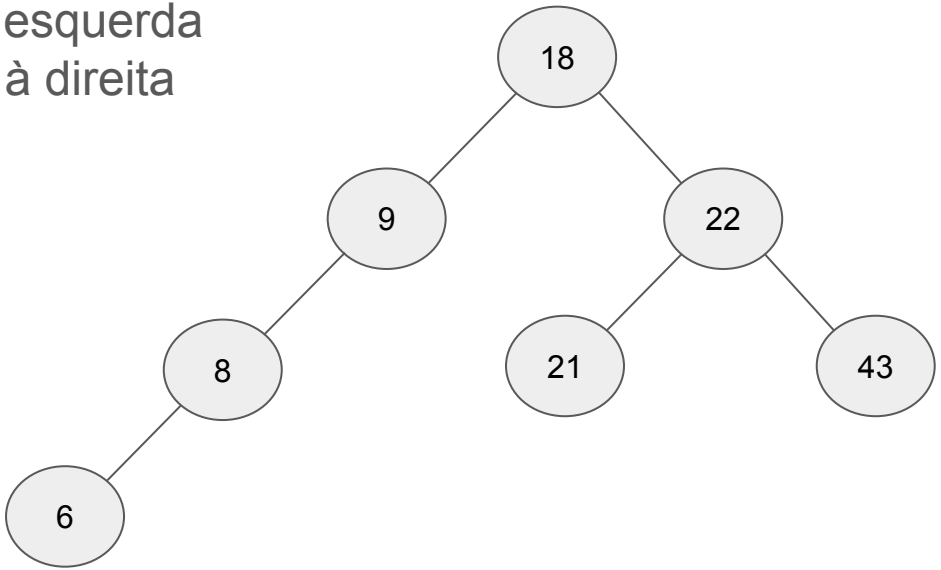
Rotação à esquerda
+ Rotação à direita



Balanceamento Dinâmico por Rotação

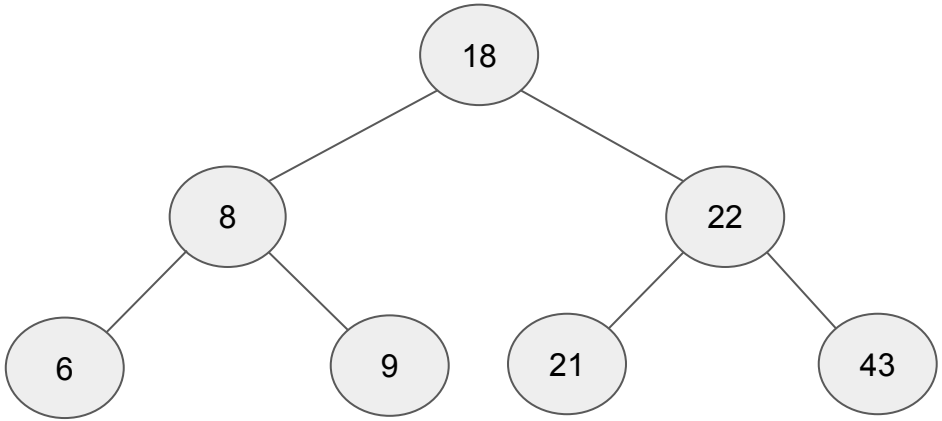
- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6 e 8
- Com balanceamento:

Rotação à esquerda
+ Rotação à direita



Balanceamento Dinâmico por Rotação

- Nós inseridos na ordem: 43, 18, 22, 9, 21, 6 e 8
- Com balanceamento:



Resumo

- É possível definir uma ordem sobre os elementos de uma árvore.
- O desempenho depende da altura da árvore de busca.
- Balanceamento (estático/dinâmico) ajusta a estrutura da árvore para manter o bom desempenho.